

Operating Systems - Chapter 26 Summary

Concurrency: An Introduction

- Threads introduce concurrency within a single process, allowing multiple sequences of execution (PCs) within one address space.
- Each thread has its own registers and stack but shares the same code, heap, and global variables with other threads.
- Context switches between threads are faster than between processes since the address space remains the same.

Why Use Threads?

- Parallelism: utilize multiple CPUs to perform computations in parallel (e.g., array operations).
- Overlapping I/O and computation: when one thread blocks waiting for I/O, another can continue execution.
- Easier sharing of data than processes, since threads share the same memory address space.

Example: Thread Creation

- Threads are created using `pthread_create()` in C, which runs a given function in a new thread.
- `pthread_join()` waits for a thread to complete before proceeding.
- Execution order is nondeterministic — it depends on the OS scheduler.

Shared Data and Race Conditions

- Threads share global variables, which can lead to data races if accessed concurrently without protection.
- Example: two threads incrementing a shared counter may produce incorrect results due to interleaved operations.
- Race condition: when the result of computation depends on timing or interleaving of threads.

Critical Sections and Mutual Exclusion

- A critical section is a piece of code that accesses shared resources (variables, data structures).
- Mutual exclusion ensures that only one thread executes a critical section at a time.
- Failure to enforce mutual exclusion leads to nondeterministic behavior and incorrect results.

Atomicity and Synchronization

- Atomic operations execute as a single, indivisible step: either all actions happen, or none do.
- Hardware instructions (e.g., test-and-set, compare-and-swap) enable synchronization primitives.
- Software builds higher-level synchronization tools (locks, semaphores, condition variables) using atomic hardware operations.

Waiting for Other Threads

- Threads often need to wait for events triggered by other threads (e.g., I/O completion, task synchronization).
- Mechanisms like condition variables and signals allow threads to sleep and wake efficiently.

Key Terms

- Critical section: code accessing a shared resource.
- Race condition: unexpected behavior due to concurrent access.
- Indeterminate program: output varies due to races.
- Mutual exclusion: guarantees only one thread in a critical section at a time.

Why Threads in an OS Course?

- The OS is the first concurrent program — it must manage interrupts, I/O, and shared data structures safely.
- Early OS designers developed synchronization primitives to protect kernel data (page tables, process lists, file systems).
- These concepts are foundational to both kernel and user-level concurrent programming.

Summary

Threads enable concurrency within processes, improving parallelism and responsiveness. They introduce complexity due to shared memory, leading to race conditions and the need for synchronization. Mutual exclusion, atomicity, and synchronization primitives form the foundation of safe concurrent programming. Understanding these mechanisms is crucial for OS design and modern multithreaded applications.